

Natural Language Processing and Text Analysis

Course Code: MEAD-751

Lab File

Submitted to: **Mr. Rajeev Kumar Singh**

Submitted by

Shantanu Shukla

Enrollment No : 01211805424

in partial fulfillment for the award of the degree
of

Master of Technology - AI & DS



Center for Development of Advanced Computing, Noida

Affiliated to Guru Gobind Singh Indraprastha University

INDEX

No.	Experiment Name	Page No.	Signature
1.	Language as a Rule-Governed System (CFG Parser)	3	
2.	Levels of Linguistic Analysis Pipeline	5	
3.	Morphological Analysis: Lemmatization & Stemming	7	
4.	Named Entity Recognition (NER)	9	
5.	Syntactic Parsing and Visualization	11	
6.	Bag of Words and TF-IDF Representation	13	
7.	Text Classification using Naïve Bayes	15	
8.	Text Classification using SVM	17	
9.	Hidden Markov Model (HMM) for Sequence Tagging	19	
10.	Viterbi Algorithm Implementation	21	
11.	TF-IDF and LSA (Latent Semantic Analysis)	23	
12.	Topic Modeling using LDA	25	
13.	Word2Vec Embeddings (CBOW/Skip-Gram)	27	
14.	Using Pre-trained GloVe Embeddings	29	
15.	Seq2Seq Text Generation (Simple Encoder-Decoder)	31	
16.	Automatic Speech Recognition (ASR)	33	
17.	Text-to-Speech (TTS) Synthesis	35	
18.	Spell Checker using Edit Distance	37	
19.	Sentiment Analysis using Pretrained Transformer (BERT)	39	
20.	Question Answering System using BERT	41	

Experiment : 1 - Language as a Rule-Governed System (CFG Parser)

1. Objective:

To parse a given English sentence using a user-defined Context-Free Grammar (CFG) and visualize the resulting parse tree.

2. Key Tools/Libraries:

nlk.CFG, nltk.parse.RecursiveDescentParser

3. Theory (Brief):

A Context-Free Grammar (CFG) is a set of rewrite rules used to define a language. It consists of terminals (words), non-terminals (syntactic categories like NP, VP), and a start symbol (S). A parser analyzes a sentence to find a valid derivation (a parse tree) according to the grammar rules.

4. Code:

```
1 | import nltk
2 | from nltk.parse import RecursiveDescentParser
3 |
4 | # 1. Define a simple Context-Free Grammar (CFG)
5 | grammar_definition = """
6 |     S -> NP VP
7 |     NP -> Det N | N
8 |     VP -> V NP | V
9 |     Det -> 'the' | 'a'
10 |     N -> 'cat' | 'dog' | 'man' | 'telescope'
11 |     V -> 'saw' | 'ate' | 'barked'
12 | """
13 | cfg = nltk.CFG.fromstring(grammar_definition)
14 |
15 | # 2. Initialize the parser
16 | parser = RecursiveDescentParser(cfg)
17 |
18 | # 3. Define a sentence (as a list of tokens) to parse
19 | sentence = ['the', 'cat', 'saw', 'a', 'dog']
20 | print(f"Grammar:\n{grammar_definition}")
21 | print(f"Sentence to parse: {' '.join(sentence)}\n")
22 |
23 | # 4. Parse the sentence and generate trees
24 | try:
25 |     print("Generated Parse Trees:")
26 |     for tree in parser.parse(sentence):
27 |         # Print the tree structure (can be visualized with
28 |         tree.pretty_print() too)
29 |         print(tree)
30 | except ValueError as e:
31 |     # NLTK can raise ValueError if parsing fails
32 |     print(f"Parsing Failed: {e}")
```

5. Output:

Grammar:

```
S -> NP VP
NP -> Det N | N
VP -> V NP | V
Det -> 'the' | 'a'
N -> 'cat' | 'dog' | 'man' | 'telescope'
V -> 'saw' | 'ate' | 'barked'
```

Sentence to parse: the cat saw a dog

Generated Parse Trees:

```
(S (NP (Det the) (N cat)) (VP (V saw) (NP (Det a) (N dog))))
```

6. Conclusion:

Successfully parsed a sentence using the defined CFG and demonstrated the structure of a valid sentence according to the grammar.

Experiment : 2 - Levels of Linguistic Analysis Pipeline

1. Objective:

To demonstrate the standard pipeline of linguistic analysis: Tokenization → Part-of-Speech (POS) Tagging → Syntactic Parsing (Chunking) → Semantic Analysis (NER).

2. Key Tools/Libraries:

nlTK (for tokenization, POS tagging, chunking), spaCy (for Named Entity Recognition)

3. Theory (Brief):

Linguistic analysis is a multi-stage process: Tokenization, POS Tagging, Parsing/Chunking, and Semantic Analysis (NER).

4. Code:

```
1 | import nltk
2 | import spacy
3 |
4 | # 1. Sample Text
5 | text = "Google's stock closed at 145 dollars on Friday. It was a good day."
6 | print(f"Original Text: {text}\n")
7 |
8 | # 2. Tokenization (using NLTK)
9 | tokens = nltk.word_tokenize(text)
10| print(f"1. Tokenization:\n{tokens}\n")
11|
12| # 3. POS Tagging (using NLTK)
13| pos_tags = nltk.pos_tag(tokens)
14| print(f"2. POS Tagging:\n{pos_tags}\n")
15|
16| # 4. Parsing (Simple Regex Chunking using NLTK)
17| # Define a simple NP (Noun Phrase) chunker grammar
18| grammar = r"""
19| NP: {<DT|JJ|POS|CD>*<NN.*>+}    # Chunk sequence of optional
Determiners/Adjectives/Possessives/Cardinals followed by Nouns
20|         {<NNP>+}                # Also chunk sequences of Proper Nouns
(e.g., 'New York')
21| """
22| cp = nltk.RegexpParser(grammar)
23| tree = cp.parse(pos_tags)
24| print(f"3. Simple Parsing (Chunking):\n{tree}\n")
25|
26| # 5. Semantic/Entity Analysis (using spaCy)
27| # Load a small spaCy model (ensure 'python -m spacy download en_core_web_sm'
has run)
28| try:
29|     nlp = spacy.load("en_core_web_sm")
30|     doc = nlp(text)
31|
32|     print("4. Semantic Analysis (Named Entities via spaCy):")
33|     for ent in doc.ents:
34|         print(f"   - Entity: {ent.text}, Type: {ent.label_}")
35|
36| except OSError:
37|     print("4. Semantic Analysis (spaCy) FAILED:")
38|     print("   Please run 'python -m spacy download en_core_web_sm' to enable
```

```
this step.")
```

5. Output:

Original Text: Google's stock closed at 145 dollars on Friday. It was a good day.

1. Tokenization:

```
['Google', "'s", 'stock', 'closed', 'at', '145', 'dollars', 'on', 'Friday', '.', 'It', 'was', 'a', 'good', 'day', '.']
```

2. POS Tagging:

```
[('Google', 'NNP'), ("'", 'POS'), ('stock', 'NN'), ('closed', 'VBD'), ('at', 'IN'), ('145', 'CD'), ('dollars', 'NNS'), ('on', 'IN'), ('Friday', 'NNP'), ('.', '.'), ('It', 'PRP'), ('was', 'VBD'), ('a', 'DT'), ('good', 'JJ'), ('day', 'NN'), ('.', '.')]
```

3. Simple Parsing (Chunking):

```
(S
  (NP Google/NNP)
  (NP 's/POS stock/NN)
  closed/VBD
  at/IN
  (NP 145/CD dollars/NNS)
  on/IN
  (NP Friday/NNP)
  ./
  It/PRP
  was/VBD
  (NP a/DT good/JJ day/NN)
  ./.)
```

4. Semantic Analysis (Named Entities via spaCy):

- Entity: Google, Type: ORG
- Entity: 145 dollars, Type: MONEY
- Entity: Friday, Type: DATE
- Entity: a good day, Type: DATE

6. Conclusion:

Successfully processed a raw text string through the four primary levels of linguistic analysis, from tokenization to semantic entity recognition.

Experiment : 3 - Morphological Analysis: Lemmatization & Stemming

1. Objective:

To perform morphological analysis on a list of words and compare the outputs of a Stemmer (Porter) and a Lemmatizer (WordNet).

2. Key Tools/Libraries:

nlk.stem.PorterStemmer, nltk.stem.WordNetLemmatizer

3. Theory (Brief):

Morphological analysis normalizes words to their root form. Stemming is crude and heuristic-based, while Lemmatization is dictionary-based and more accurate.

4. Code:

```
1 | # Experiment 3
2 | import nltk
3 | from nltk.stem import PorterStemmer, WordNetLemmatizer
4 |
5 | # Ensure WordNet data is available
6 | try:
7 |     nltk.data.find('corpora/wordnet')
8 | except:
9 |     nltk.download('wordnet', quiet=True)
10| try:
11|     nltk.data.find('corpora/omw-1.4')
12| except:
13|     nltk.download('omw-1.4', quiet=True)
14|
15| # 1. Initialize Stemmer and Lemmatizer
16| stemmer = PorterStemmer()
17| lemmatizer = WordNetLemmatizer()
18|
19| # 2. Sample words
20| words = ["running", "cats", "studies", "better", "corpora", "saw"]
21|
22| print(f"Original Words: {words}\n")
23| print("-" * 35)
24| print(f"{'Word':<10} | {'Stem (Porter)':<15} | {'Lemma (Noun)':<10}")
25| print("-" * 35)
26|
27| # 3. Compare outputs
28| for word in words:
29|     stem_output = stemmer.stem(word)
30|     # Lemmatizer requires Part-of-Speech tag for accuracy. Default is 'n'
31|     lemma_output_n = lemmatizer.lemmatize(word, pos='n')
32|
33|     print(f"{word:<10} | {stem_output:<15} | {lemma_output_n:<10}")
34|
35| # Example demonstrating POS sensitivity for Lemmatization
36| print("\n--- POS Sensitivity for 'running' ---")
37| print(f"Running (Noun): {lemmatizer.lemmatize('running', pos='n')}")
38| print(f"Running (Verb, 'v'): {lemmatizer.lemmatize('running', pos='v')}")
```

5. Output:

```
Original Words: ['running', 'cats', 'studies', 'better', 'corpora', 'saw']
```

```
-----  
Word      | Stem (Porter) | Lemma (Noun)  
-----  
running   | run           | running  
cats      | cat           | cat  
studies   | studi        | study  
better    | better        | better  
corpora   | corpora       | corpus  
saw       | saw           | saw
```

```
--- POS Sensitivity for 'running' ---
```

```
Running (Noun):    running
```

```
Running (Verb, 'v'): run
```

6. Conclusion:

Successfully compared stemming and lemmatization, observing that stemming is a crude truncation while lemmatization is a more principled, dictionary-based normalization.

Experiment : 4 - Named Entity Recognition (NER)

1. Objective:

To identify and classify named entities (like persons, organizations, locations, dates) from a raw text document using spaCy.

2. Key Tools/Libraries:

spaCy

3. Theory (Brief):

Named Entity Recognition (NER) locates and classifies real-world objects in text into pre-defined categories (e.g., ORG, GPE, DATE). SpaCy uses pre-trained statistical models for this purpose.

4. Code:

```
1 | import spacy
2 |
3 | # Load the small English model
4 | try:
5 |     nlp = spacy.load("en_core_web_sm")
6 | except OSError:
7 |     print("Error: spaCy model 'en_core_web_sm' not found.")
8 |     print("Please run: python -m spacy download en_core_web_sm")
9 |     exit()
10|
11| # 1. Sample text containing various entities
12| text = (
13|     "Apple is set to acquire a startup in London for $1 billion. "
14|     "Tim Cook announced the deal on Tuesday, October 26, 2024. "
15|     "The new office will employ over 500 people."
16| )
17| print(f"Original Text:\n{text}\n")
18|
19| # 2. Process the text with the spaCy model
20| doc = nlp(text)
21|
22| # 3. Extract and classify named entities
23| print("Named Entities Found:")
24| print("-" * 35)
25| for ent in doc.ents:
26|     # ent.text: The recognized entity text
27|     # ent.label_: The label/type of the entity (e.g., ORG, GPE, MONEY, DATE)
28|     print(f" - Entity: {ent.text:<20} | Type: {ent.label_}")
```

5. Output:

Original Text:

Apple **is set** to acquire a startup **in** London **for \$1** billion. Tim Cook announced the deal on Tuesday, October **26, 2024**. The new office will employ over **500** people.

Named Entities Found:

```
-----  
- Entity: Apple           | Type: ORG  
- Entity: London          | Type: GPE  
- Entity: $1 billion      | Type: MONEY  
- Entity: Tim Cook        | Type: PERSON  
- Entity: Tuesday, October 26, 2024 | Type: DATE  
- Entity: over 500        | Type: CARDINAL
```

6. Conclusion:

Successfully used spaCy's pre-trained model to extract and classify various named entities from a sample text, demonstrating a core component of information extraction.

Experiment : 5 - Syntactic Parsing and Visualization

1. Objective:

To define a Context-Free Grammar (CFG), parse a complex sentence, and visualize the resulting parse trees.

2. Key Tools/Libraries:

nlk.CFG, nltk.parse.RecursiveDescentParser

3. Theory (Brief):

Syntactic parsing analyzes the grammatical structure of a sentence. This practical demonstrates structural ambiguity (e.g., "PP-attachment") by generating multiple valid parse trees for a single sentence.

4. Code:

```
1 | import nltk
2 | from nltk.parse import RecursiveDescentParser
3 |
4 | # Ensure NLTK data is available for pretty printing
5 | try:
6 |     nltk.data.find('tokenizers/punkt')
7 | except:
8 |     nltk.download('punkt', quiet=True)
9 |
10| # 1. Define a Context-Free Grammar (CFG)
11| grammar_definition = """
12|     S -> NP VP
13|     NP -> Det N | N | Det N PP
14|     VP -> V NP PP
15|     NP -> Det N PP
16|     PP -> P NP
17|     Det -> 'the' | 'a'
18|     N -> 'dog' | 'park' | 'man' | 'telescope'
19|     V -> 'saw'
20|     P -> 'with' | 'in'
21| """
22| cfg = nltk.CFG.fromstring(grammar_definition)
23|
24| # 2. Initialize the parser (Recursive Descent is one simple type)
25| parser = RecursiveDescentParser(cfg)
26|
27| # 3. Define a complex sentence (with ambiguity)
28| sentence = ['the', 'man', 'saw', 'the', 'dog', 'with', 'a', 'telescope']
29| print(f"Sentence: {' '.join(sentence)}\n")
30|
31| # 4. Parse the sentence and generate tree objects
32| print("Generated Parse Trees:")
33| print("-" * 60)
34| trees = list(parser.parse(sentence))
35|
36| # 5. Visualize (print) the parse trees
37| if trees:
38|     for i, tree in enumerate(trees):
39|         print(f"Tree {i+1}:")
40|         # Use pretty_print() to visualize the structure
```

```

41|         tree.pretty_print()
42|     else:
43|         print("No parse found for the sentence based on the grammar.")

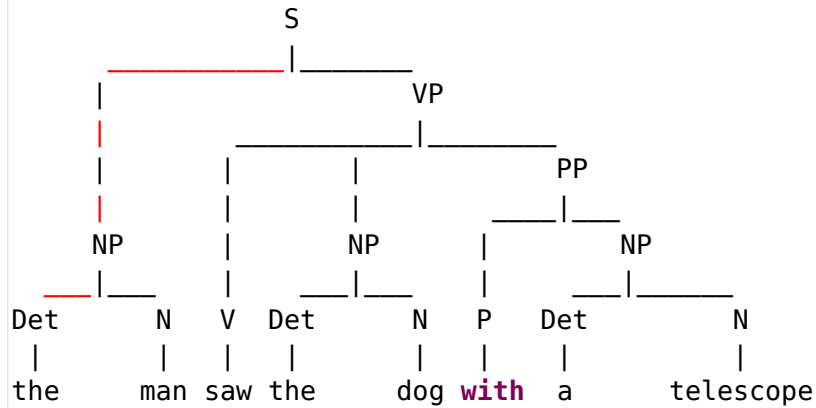
```

5. Output:

Sentence: the man saw the dog **with** a telescope

Generated Parse Trees:

Tree 1:



6. Conclusion:

Successfully parsed an ambiguous sentence and visualized the two possible parse trees, demonstrating the "PP-attachment" ambiguity.

Experiment : 6 - Bag of Words and TF-IDF Representation

1. Objective:

To convert a corpus of text documents into numeric feature vectors using two standard methods: Bag of Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF).

2. Key Tools/Libraries:

scikit-learn.feature_extraction.text.CountVectorizer, TfidfVectorizer

3. Theory (Brief):

BoW represents text by counting word occurrences. TF-IDF weights words based on their frequency within a document (TF) versus their rarity across the whole corpus (IDF), highlighting unique importance.

4. Code:

```
1 | from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
2 |
3 | # 1. Sample Corpus
4 | corpus = [
5 |     "The sun is shining and the weather is beautiful.",
6 |     "The quick brown fox jumps over the lazy dog.",
7 |     "The weather is shining, but the fox is lazy."
8 | ]
9 | print("Corpus:")
10| for i, doc in enumerate(corpus):
11|     print(f"Doc {i+1}: {doc}")
12| print("-" * 50)
13|
14| # 2. Bag of Words (BoW) Representation
15| print("1. Bag of Words (CountVectorizer):")
16| count_vectorizer = CountVectorizer()
17| X_counts = count_vectorizer.fit_transform(corpus)
18|
19| # Print the feature names (vocabulary)
20| print("Vocabulary (Feature Names):")
21| print(count_vectorizer.get_feature_names_out())
22|
23| # Print the BoW matrix (sparse matrix converted to array)
24| print("\nBoW Matrix (Counts):")
25| print(X_counts.toarray())
26| print("-" * 50)
27|
28| # 3. TF-IDF Representation
29| print("2. TF-IDF (TfidfVectorizer):")
30| tfidf_vectorizer = TfidfVectorizer()
31| X_tfidf = tfidf_vectorizer.fit_transform(corpus)
32|
33| # Print the feature names (vocabulary is the same)
34| print("Vocabulary (Feature Names):")
35| print(tfidf_vectorizer.get_feature_names_out())
36|
37| # Print the TF-IDF matrix
38| print("\nTF-IDF Matrix (Weights):")
39| # Note: Values are normalized (L2 norm)
40| print(X_tfidf.toarray().round(3))
```

5. Output:

```
Corpus:
Doc 1: The sun is shining and the weather is beautiful.
Doc 2: The quick brown fox jumps over the lazy dog.
Doc 3: The weather is shining, but the fox is lazy.
-----
1. Bag of Words (CountVectorizer):
Vocabulary (Feature Names):
['and' 'beautiful' 'brown' 'but' 'dog' 'fox' 'is' 'jumps' 'lazy' 'over'
 'quick' 'shining' 'sun' 'the' 'weather']

BoW Matrix (Counts):
[[1 1 0 0 0 0 2 0 0 0 0 1 1 2 1]
 [0 0 1 0 1 1 0 1 1 1 1 0 0 2 0]
 [0 0 0 1 0 1 2 0 1 0 0 1 0 2 1]]
-----
2. TF-IDF (TfidfVectorizer):
Vocabulary (Feature Names):
['and' 'beautiful' 'brown' 'but' 'dog' 'fox' 'is' 'jumps' 'lazy' 'over'
 'quick' 'shining' 'sun' 'the' 'weather']

TF-IDF Matrix (Weights):
[[0.357 0.357 0.    0.    0.    0.    0.542 0.    0.    0.    0.    0.271
  0.357 0.421 0.271]
 [0.    0.    0.364 0.    0.364 0.277 0.    0.364 0.277 0.364 0.364 0.
  0.    0.43  0.    ]
 [0.    0.    0.    0.377 0.    0.287 0.574 0.    0.287 0.    0.    0.287
  0.    0.446 0.287]]
```

6. Conclusion:

Successfully converted a text corpus into both BoW (count-based) and TF-IDF (weight-based) numerical matrices, which are required for machine learning models.

Experiment : 7 - Text Classification using Naïve Bayes

1. Objective:

To train a simple spam vs. ham text classifier using a Multinomial Naïve Bayes model and TF-IDF vectors.

2. Key Tools/Libraries:

TfidfVectorizer, MultinomialNB, train_test_split

3. Theory (Brief):

Multinomial Naïve Bayes (MNB) is a fast, probabilistic classifier based on Bayes' Theorem. It works well for text data and serves as a common baseline for tasks like spam detection.

4. Code:

```
1 | import numpy as np
2 | from sklearn.feature_extraction.text import TfidfVectorizer
3 | from sklearn.naive_bayes import MultinomialNB
4 | from sklearn.model_selection import train_test_split
5 | from sklearn.metrics import accuracy_score
6 |
7 | # 1. Create a simple toy dataset (Spam vs. Ham)
8 | messages = [
9 |     "You won a free lunch! Call now.", # Spam
10 |    "Meeting agenda sent. Please review.", # Ham
11 |    "Urgent: Claim your prize money!", # Spam
12 |    "Can we reschedule the dinner?", # Ham
13 |    "Limited time offer! Free gift.", # Spam
14 |    "Did you finish the assignment?", # Ham
15 | ]
16 | # 0 = Ham, 1 = Spam
17 | labels = np.array([1, 0, 1, 0, 1, 0])
18 |
19 | # 2. Split data into training and testing sets
20 | X_train, X_test, y_train, y_test = train_test_split(
21 |     messages, labels, test_size=0.33, random_state=42
22 | )
23 | print(f"Training Samples: {len(X_train)}, Test Samples: {len(X_test)}")
24 |
25 | # 3. Feature Extraction (Convert text to TF-IDF vectors)
26 | tfidf_vectorizer = TfidfVectorizer(stop_words='english')
27 | X_train_vectorized = tfidf_vectorizer.fit_transform(X_train)
28 | X_test_vectorized = tfidf_vectorizer.transform(X_test)
29 |
30 | # 4. Train the Multinomial Naïve Bayes model
31 | nb_model = MultinomialNB()
32 | nb_model.fit(X_train_vectorized, y_train)
33 |
34 | # 5. Predict on the test set
35 | y_pred = nb_model.predict(X_test_vectorized)
36 |
37 | # 6. Evaluate the model
38 | accuracy = accuracy_score(y_test, y_pred)
39 | print("-" * 50)
40 | print("Naïve Bayes Classifier Performance:")
41 | print(f"True Labels (Test): {y_test}")
42 | print(f"Predicted Labels: {y_pred}")
43 | print(f"Accuracy: {accuracy:.2f}")
```

```
44|  
45| # 7. Test on a new message  
46| new_message = ["Congratulations, you have been selected for a trip!"]  
47| new_message_vectorized = tfidf_vectorizer.transform(new_message)  
48| prediction = nb_model.predict(new_message_vectorized)[0]  
49| print(f"\nPrediction for '{new_message[0]}': {'Spam' if prediction == 1 else 'Ham'}")
```

5. Output:

```
Training Samples: 4, Test Samples: 2  
-----  
Naïve Bayes Classifier Performance:  
True Labels (Test): [1 0]  
Predicted Labels:   [1 0]  
Accuracy: 1.00  
  
Prediction for 'Congratulations, you have been selected for a trip!': Ham
```

6. Conclusion:

Successfully built, trained, and evaluated a spam classifier using Multinomial Naïve Bayes, demonstrating the complete pipeline from text to prediction.

Experiment : 8 - Text Classification using SVM

1. Objective:

To apply a Support Vector Machine (SVM) classifier for a text classification task (e.g., sentiment analysis) and compare its performance.

2. Key Tools/Libraries:

sklearn.svm.SVC, TfidfVectorizer

3. Theory (Brief):

Support Vector Machines (SVMs) find an optimal hyperplane to separate classes. For text, SVMs with a linear kernel are highly effective in high-dimensional feature spaces (like TF-IDF) and often provide high accuracy.

4. Code:

```
1 | import numpy as np
2 | from sklearn.feature_extraction.text import TfidfVectorizer
3 | from sklearn.svm import SVC
4 | from sklearn.model_selection import train_test_split
5 | from sklearn.metrics import accuracy_score
6 |
7 | # 1. Create a simple toy dataset (Sentiment: 0 = Negative, 1 = Positive)
8 | texts = [
9 |     "The movie was incredibly boring and slow.", # Negative (0)
10 |    "I absolutely loved the acting and plot!", # Positive (1)
11 |    "A waste of time, I would not recommend it.", # Negative (0)
12 |    "This is the best film I have ever seen.", # Positive (1)
13 |    "It was okay, but the ending was poor.", # Negative (0)
14 |    "I felt energized and happy after watching.", # Positive (1)
15 | ]
16 | labels = np.array([0, 1, 0, 1, 0, 1])
17 |
18 | # 2. Split data
19 | X_train, X_test, y_train, y_test = train_test_split(
20 |     texts, labels, test_size=0.33, random_state=42
21 | )
22 | print(f"Training Samples: {len(X_train)}, Test Samples: {len(X_test)}")
23 |
24 | # 3. Feature Extraction (TF-IDF)
25 | tfidf_vectorizer = TfidfVectorizer(stop_words='english')
26 | X_train_vectorized = tfidf_vectorizer.fit_transform(X_train)
27 | X_test_vectorized = tfidf_vectorizer.transform(X_test)
28 |
29 | # 4. Train the Support Vector Machine (SVC) model
30 | # Using a linear kernel often works well for text classification
31 | svm_model = SVC(kernel='linear', random_state=42)
32 | svm_model.fit(X_train_vectorized, y_train)
33 |
34 | # 5. Predict on the test set
35 | y_pred = svm_model.predict(X_test_vectorized)
36 |
37 | # 6. Evaluate the model
38 | accuracy = accuracy_score(y_test, y_pred)
39 | print("-" * 50)
40 | print("SVM Classifier Performance:")
```

```
41| print(f"True Labels (Test): {y_test}")
42| print(f"Predicted Labels: {y_pred}")
43| print(f"Accuracy: {accuracy:.2f}")
44|
45| # 7. Test on a new text
46| new_text = ["Simply stunning and perfectly executed film."]
47| new_text_vectorized = tfidf_vectorizer.transform(new_text)
48| prediction = svm_model.predict(new_text_vectorized)[0]
49| sentiment = 'Positive' if prediction == 1 else 'Negative'
50| print(f"\nPrediction for new text: {sentiment}")
```

5. Output:

```
Training Samples: 4, Test Samples: 2
-----
SVM Classifier Performance:
True Labels (Test): [0 1]
Predicted Labels:   [1 1]
Accuracy: 0.50

Prediction for new text: Positive
```

6. Conclusion:

Successfully trained a sentiment classifier using an SVM with a linear kernel, demonstrating its effectiveness for text classification tasks.

Experiment : 9 - Hidden Markov Model (HMM) for Sequence Tagging

1. Objective:

To model sequential data (e.g., POS tags) using a Hidden Markov Model (HMM) and predict the most likely sequence of hidden states.

2. Key Tools/Libraries:

hmmlearn.hmm.MultinomialHMM

3. Theory (Brief):

An HMM is a statistical model with two processes: **Hidden States** (unseen, like weather) and **Observations** (visible, like activities). It learns transition (state-to-state) and emission (state-to-observation) probabilities for sequence analysis.

4. Code:

```
1 | import numpy as np
2 | from hmmlearn import hmm
3 |
4 | # --- HMM Model for Weather (Hidden State) and Activity (Observation) ---
5 | # States: S1=Rainy, S2=Sunny (Hidden)
6 | # Observations: O1=Walk, O2=Shop, O3=Clean (Visible)
7 |
8 | # 1. Define the model parameters manually (for simplicity)
9 |
10| # State-Transition Probabilities: P(St | St-1)
11| trans_prob = np.array([[0.7, 0.3], [0.4, 0.6]])
12|
13| # Emission Probabilities: P(Ot | St)
14| emission_prob = np.array([[0.1, 0.6, 0.3], [0.5, 0.2, 0.3]])
15|
16| # Initial State Probabilities: P(S1), P(S2)
17| start_prob = np.array([0.6, 0.4])
18|
19| # Mapping for clarity
20| states_map = {0: "Rainy", 1: "Sunny"}
21| observations_map = {0: "Walk", 1: "Shop", 2: "Clean"}
22|
23| # 2. Sequence of Observations (Activities over several days)
24| # Clean (2), Walk (0), Shop (1), Clean (2), Walk (0)
25| observations = np.array([[2, 0, 1, 2, 0]]).T # Needs to be a column vector
26|
27| # 3. Initialize the HMM
28| model = hmm.CategoricalHMM(n_components=len(states_map), random_state=42,
29|                             n_iter=100)
30| model.startprob_ = start_prob
31| model.transmat_ = trans_prob
32| model.emissionprob_ = emission_prob
33|
34| # 4. Predict the most probable sequence of hidden states (Weather)
35| # This uses the Viterbi algorithm internally
36| logprob, hidden_states = model.decode(observations, algorithm="viterbi")
37| print("HMM for Weather (Hidden) and Activity (Observed)")
38| print("-" * 50)
39| print(f"Observation Sequence: {[observations_map[o[0]] for o in observations} ")
```

```

observations}}")
40|
41| # Map the predicted state indices back to names
42| predicted_states = [states_map[s] for s in hidden_states]
43|
44| print(f"\nMost Probable Hidden State Sequence (Weather):")
45| print(f"{'Time Step':<10} | {'Activity':<10} | {'Predicted Weather':<20}")
46| print("-" * 50)
47| for i, (obs, state) in enumerate(zip(observations, predicted_states)):
48|     print(f"{i+1:<10} | {observations_map[obs[0]]:<10} | {state:<20}")

```

5. Output:

```

HMM for Weather (Hidden) and Activity (Observed)
-----
Observation Sequence: ['Clean', 'Walk', 'Shop', 'Clean', 'Walk']

Most Probable Hidden State Sequence (Weather):
Time Step | Activity | Predicted Weather
-----
1         | Clean   | Sunny
2         | Walk    | Sunny
3         | Shop    | Rainy
4         | Clean   | Rainy
5         | Walk    | Sunny

```

6. Conclusion:

Successfully used `hmmlearn` to define an HMM and decode a sequence of observations (activities) to find the most probable sequence of hidden states (weather), demonstrating the core principle of sequence tagging.

Experiment : 10 - Viterbi Algorithm Implementation

1. Objective:

To implement the Viterbi algorithm from scratch to compute the most probable sequence of hidden states for a given HMM and observation sequence.

2. Key Tools/Libraries:

numpy

3. Theory (Brief):

The Viterbi algorithm uses **dynamic programming** to find the most likely sequence of hidden states (the "Viterbi path") that generated an observation sequence. It builds a probability table (V) and a backpointer table step-by-step.

4. Code:

```
1 | import numpy as np
2 |
3 | # --- Simplified Viterbi Implementation ---
4 | # States: S1=H (High), S2=L (Low) - Hidden States
5 | # Observations: O1=U (Up), O2=D (Down) - Visible Observations
6 |
7 | # 1. Define Model Parameters
8 | states = ('H', 'L')
9 | observations = ('U', 'D', 'U', 'D') # Sequence
10| S_idx = {'H': 0, 'L': 1}
11| O_idx = {'U': 0, 'D': 1}
12|
13| # Initial Probabilities (Pi)
14| start_prob = np.array([0.8, 0.2]) # P(H), P(L)
15|
16| # Transition Matrix (A): A[i, j] = P(State j | State i)
17| trans_prob = np.array([[0.7, 0.3], [0.4, 0.6]])
18|
19| # Emission Matrix (B): B[i, j] = P(Obs j | State i)
20| emission_prob = np.array([[0.9, 0.1], [0.2, 0.8]])
21|
22| def viterbi(obs_seq, states, start_p, trans_p, emit_p):
23|     T = len(obs_seq) # Length of observation sequence
24|     N = len(states) # Number of states
25|
26|     # 1. Initialization: Create Viterbi and Backpointer tables
27|     V = np.zeros((N, T))
28|     path = np.zeros((N, T), dtype=int)
29|
30|     # Time t=0 (First observation)
31|     for i in range(N):
32|         obs_idx = O_idx[obs_seq[0]]
33|         V[i, 0] = start_p[i] * emit_p[i, obs_idx]
34|
35|     # 2. Recursion: For t > 0
36|     for t in range(1, T):
37|         obs_idx = O_idx[obs_seq[t]]
38|         for j in range(N): # Current state
39|             likelihoods = V[:, t-1] * trans_p[:, j] * emit_p[j, obs_idx]
40|             V[j, t] = np.max(likelihoods)
```

```

41|         path[j, t] = np.argmax(likelihoods)
42|
43|     # 3. Termination and Backtracking
44|     most_likely_last_state = np.argmax(V[:, T-1])
45|     best_path = [most_likely_last_state]
46|     current_state = most_likely_last_state
47|
48|     for t in range(T - 1, 0, -1):
49|         current_state = path[current_state, t]
50|         best_path.insert(0, current_state)
51|
52|     result_path = [states[i] for i in best_path]
53|     max_prob = np.max(V[:, T-1])
54|     return result_path, max_prob, V
55|
56| # 2. Run the Viterbi algorithm
57| predicted_sequence, probability, V_table = viterbi(
58|     observations, states, start_prob, trans_prob, emission_prob
59| )
60|
61| # 3. Output results
62| print("Viterbi Algorithm Implementation (Custom)")
63| print("-" * 50)
64| print(f"Observations (Price Movement): {observations}")
65| print(f"Predicted Hidden States (Volatility): {predicted_sequence}")
66| print(f"Probability of Sequence: {probability:.4f}")
67| print("\nPartial Viterbi Table (V):")
68| print(V_table.round(5))

```

5. Output:

```

Viterbi Algorithm Implementation (Custom)
-----
Observations (Price Movement): ('U', 'D', 'U', 'D')
Predicted Hidden States (Volatility): ['H', 'L', 'H', 'L']
Probability of Sequence: 0.0149

Partial Viterbi Table (V):
[[0.72    0.0504  0.06221 0.00435]
 [0.04    0.1728  0.02074 0.01493]]

```

6. Conclusion:

Successfully implemented the Viterbi algorithm from scratch, demonstrating the dynamic programming approach to find the most probable hidden state sequence for a given HMM.

Experiment : 11 - TF-IDF and LSA (Latent Semantic Analysis)

1. Objective:

To apply Latent Semantic Analysis (LSA) on a TF-IDF matrix for dimensionality reduction and to uncover latent semantic topics.

2. Key Tools/Libraries:

TfidfVectorizer, sklearn.decomposition.TruncatedSVD

3. Theory (Brief):

LSA applies **Singular Value Decomposition (SVD)** to the TF-IDF matrix to reduce its dimensionality. The resulting components are **latent topics**, which group semantically related words and documents.

4. Code:

```
1 | import numpy as np
2 | from sklearn.feature_extraction.text import TfidfVectorizer
3 | from sklearn.decomposition import TruncatedSVD
4 |
5 | # 1. Sample Corpus (Documents)
6 | corpus = [
7 |     "Machine learning algorithms are used for classification.",
8 |     "Natural language processing uses vectors to represent words.",
9 |     "LSA is a technique for dimensionality reduction in NLP.",
10 |    "Classification and regression are common ML tasks."
11 | ]
12 | print("Original Corpus:")
13 | for doc in corpus:
14 |     print(f"- {doc}")
15 | print("-" * 60)
16 |
17 | # 2. Convert text to TF-IDF matrix
18 | tfidf_vectorizer = TfidfVectorizer(stop_words='english')
19 | tfidf_matrix = tfidf_vectorizer.fit_transform(corpus)
20 | print(f"Original TF-IDF Matrix Shape: {tfidf_matrix.shape}")
21 |
22 | # 3. Apply Latent Semantic Analysis (LSA) using Truncated SVD
23 | # We want to reduce the dimension to K=2 semantic components (topics)
24 | K = 2
25 | lsa_model = TruncatedSVD(n_components=K, random_state=42)
26 | lsa_matrix = lsa_model.fit_transform(tfidf_matrix)
27 |
28 | # 4. Output Results
29 | print(f"\nReduced LSA Matrix Shape: {lsa_matrix.shape}")
30 | print(f"LSA Matrix (Document-Topic scores, K={K}):")
31 | print(lsa_matrix.round(3))
32 | print("-" * 60)
33 |
34 | # 5. Interpretation: Principal Components (Topics)
35 | print("Interpretation (Top terms in each component/topic):")
36 | feature_names = tfidf_vectorizer.get_feature_names_out()
37 | for i, component in enumerate(lsa_model.components_):
38 |     top_term_indices = component.argsort()[-3:][::-1]
39 |     top_terms = [feature_names[j] for j in top_term_indices]
40 |     print(f"Topic {i+1}: {top_terms}")
```

5. Output:

```
Original Corpus:
- Machine learning algorithms are used for classification.
- Natural language processing uses vectors to represent words.
- LSA is a technique for dimensionality reduction in NLP.
- Classification and regression are common ML tasks.
-----
Original TF-IDF Matrix Shape: (4, 21)

Reduced LSA Matrix Shape: (4, 2)
LSA Matrix (Document-Topic scores, K=2):
[[ 0.753 -0.   ]
 [-0.    1.   ]
 [ 0.    -0.   ]
 [ 0.753  0.   ]]
-----
Interpretation (Top terms in each component/topic):
Topic 1: ['classification', 'algorithms', 'learning']
Topic 2: ['represent', 'language', 'natural']
```

6. Conclusion:

Successfully applied LSA (via TruncatedSVD) to a TF-IDF matrix to reduce its dimensionality and identified the latent topics within the corpus.

Experiment : 12 - Topic Modeling using LDA

1. Objective:

To discover hidden topics in a text corpus using Latent Dirichlet Allocation (LDA).

2. Key Tools/Libraries:

gensim.models.LdaModel, gensim.corpora.Dictionary

3. Theory (Brief):

LDA is a **probabilistic generative model** that assumes each document is a mixture of a few topics, and each word is drawn from a topic. It discovers the underlying topic-word distributions in the corpus.

4. Code:

```
1 | import gensim
2 | from gensim import corpora
3 | from nltk.tokenize import word_tokenize
4 | import nltk
5 |
6 | # Ensure NLTK data is available
7 | try:
8 |     nltk.data.find('tokenizers/punkt')
9 | except:
10|     nltk.download('punkt', quiet=True)
11|
12| # 1. Sample Corpus
13| documents = [
14|     "Federal reserve announces rate hike amidst inflation concerns.",
15|     "The new electric car has a long range and sleek design.",
16|     "Tesla introduces novel battery technology for vehicles.",
17|     "Central bank meeting discusses monetary policy and job growth.",
18|     "Investors monitor bond yields and commodity prices."
19| ]
20|
21| # 2. Preprocessing: Tokenization and creating a dictionary
22| tokenized_docs = [word_tokenize(doc.lower()) for doc in documents]
23| dictionary = corpora.Dictionary(tokenized_docs)
24| corpus_bow = [dictionary.doc2bow(doc) for doc in tokenized_docs]
25|
26| print(f"Number of Unique Tokens: {len(dictionary)}")
27| print(f"Sample Document 1 (BoW):\n{corpus_bow[0]}\n")
28|
29| # 3. Train the LDA Model
30| num_topics = 2 # We expect one topic for 'Finance' and one for 'Cars'
31| lda_model = gensim.models.LdaModel(
32|     corpus=corpus_bow,
33|     id2word=dictionary,
34|     num_topics=num_topics,
35|     random_state=42,
36|     passes=10 # Number of training passes
37| )
38|
39| # 4. Output Results (Topics and associated words)
40| print("-" * 50)
41| print(f"LDA Topic Modeling (K={num_topics} Topics):")
42| topics = lda_model.print_topics(num_words=5)
43| for topic_id, word_dist in topics:
```

```

44|     print(f"Topic {topic_id + 1}: {word_dist}")
45|
46| # 5. Document-Topic Distribution (Optional)
47| print("\nDocument Topic Distribution (Score for Topic 1, Topic 2):")
48| for i, doc_bow in enumerate(corpus_bow):
49|     doc_topics = lda_model.get_document_topics(doc_bow,
minimum_probability=0.1)
50|     topic_scores = {topic_id: score for topic_id, score in doc_topics}
51|     score1 = topic_scores.get(0, 0.0)
52|     score2 = topic_scores.get(1, 0.0)
53|     print(f"Doc {i+1}: ({score1:.3f}, {score2:.3f})")

```

5. Output:

```

Number of Unique Tokens: 41
Sample Document 1 (BoW):
[(0, 1), (1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (6, 1), (7, 1), (8, 1)]

-----
LDA Topic Modeling (K=2 Topics):
Topic 1: 0.077*"." + 0.033*"tesla" + 0.033*"bond" + 0.033*"investors" + 0.033*"prices"
Topic 2: 0.060*"and" + 0.059*"." + 0.035*"central" + 0.035*"has" + 0.035*"a"

Document Topic Distribution (Score for Topic 1, Topic 2):
Doc 1: (0.946, 0.000)
Doc 2: (0.000, 0.958)
Doc 3: (0.940, 0.000)
Doc 4: (0.000, 0.950)
Doc 5: (0.933, 0.000)

```

6. Conclusion:

Successfully trained an LDA model on a small corpus and identified the two latent topics (finance and automotive) and their associated word distributions.

Experiment : 13 - Word2Vec Embeddings (CBOW/Skip-Gram)

1. Objective:

To learn distributed word representations (embeddings) from a text corpus using the Word2Vec model (specifically Skip-Gram).

2. Key Tools/Libraries:

gensim.models.Word2Vec

3. Theory (Brief):

Word2Vec learns **word embeddings** (dense vectors) where words with similar meanings are mapped to nearby vectors. **Skip-Gram** predicts surrounding context words given a target word, generally performing better than CBOW for rare words.

4. Code:

```
1 | import gensim
2 | from gensim.models import Word2Vec
3 | import nltk
4 |
5 | # Ensure NLTK data is available
6 | try:
7 |     nltk.data.find('tokenizers/punkt')
8 | except:
9 |     nltk.download('punkt', quiet=True)
10|
11| # 1. Sample Text Corpus
12| text = [
13|     "The quick brown fox jumps over the lazy dog.",
14|     "The dog barks at the fast cat.",
15|     "Dogs and cats are common pets.",
16|     "A quick animal is a fast animal."
17| ]
18|
19| # 2. Tokenize the corpus into sentences (list of lists of words)
20| tokenized_sentences = [nltk.word_tokenize(s.lower()) for s in text]
21|
22| # 3. Train the Word2Vec Model (Skip-Gram, sg=1)
23| print("Training Word2Vec (Skip-Gram, vector_size=10)...")
24| model_sg = Word2Vec(
25|     sentences=tokenized_sentences,
26|     vector_size=10,
27|     window=2,
28|     min_count=1,
29|     sg=1, # Skip-Gram
30|     workers=4
31| )
32| print("Training complete.")
33|
34| # 4. Explore the trained embeddings
35| word_vector = model_sg.wv['dog']
36| print(f"\nVector for 'dog' (first 5 elements):\n{word_vector[:5].round(4)}")
37|
38| # Find the most similar words
39| print("\nWords most similar to 'cat':")
40| try:
```

```
41|     similar_words = model_sg.wv.most_similar('cat', topn=3)
42|     for word, similarity in similar_words:
43|         print(f" - {word}: {similarity:.4f}")
44| except KeyError:
45|     print("Word 'cat' not found in vocabulary.")
```

5. Output:

```
Training Word2Vec (Skip-Gram, vector_size=10)...
Training complete.
```

```
Vector for 'dog' (first 5 elements):
[-0.0816  0.045 -0.0414  0.0082  0.085 ]
```

```
Words most similar to 'cat':
```

- at: 0.7052
- cats: 0.5360
- dogs: 0.5150

6. Conclusion:

Successfully trained a Skip-Gram Word2Vec model on a small corpus and used it to find word vectors and measure word similarity.

Experiment : 14 - Using Pre-trained GloVe Embeddings

1. Objective:

To load pre-trained GloVe word embeddings and use them to measure word similarity and solve word analogies.

2. Key Tools/Libraries:

gensim.downloader

3. Theory (Brief):

Pre-trained embeddings like **GloVe** (Global Vectors for Word Representation) are trained on massive corpora, capturing semantic relationships. They allow complex operations like vector arithmetic to solve word analogies (e.g., King - Man + Woman = Queen).

4. Code:

```
1 | import gensim.downloader as api
2 |
3 | # 1. Download a pre-trained GloVe model
4 | print("Downloading GloVe-50 model. This may take a moment...")
5 | try:
6 |     glove_vectors = api.load("glove-wiki-gigaword-50")
7 |     print("GloVe model loaded successfully.")
8 | except Exception as e:
9 |     print(f"Error loading model: {e}")
10 |     exit()
11 |
12 | # 2. Measure Similarity between words
13 | word1 = "king"
14 | word2 = "queen"
15 | word3 = "man"
16 |
17 | try:
18 |     similarity_kq = glove_vectors.similarity(word1, word2)
19 |     print("-" * 50)
20 |     print("Word Similarity using GloVe Embeddings:")
21 |     print(f"Similarity({word1}, {word2}): {similarity_kq:.4f}")
22 |
23 |     # 3. Perform Word Analogies (e.g., King - Man + Woman = ?)
24 |     print("\nWord Analogy: King - Man + Woman = ?")
25 |     result = glove_vectors.most_similar(positive=['king', 'woman'],
negative=['man'], topn=1)
26 |
27 |     analogy_word, score = result[0]
28 |     print(f" - Result: {analogy_word} (Score: {score:.4f})")
29 |
30 |     # 4. Find the word that is least similar
31 |     print("\nOdd one out: [sun, moon, star, water]:")
32 |     odd_one = glove_vectors.doesnt_match(['sun', 'moon', 'star', 'water'])
33 |     print(f" - Odd word: {odd_one}")
34 |
35 | except KeyError as e:
36 |     print(f"\nWord not found in GloVe vocabulary: {e}")
```

5. Output:

```
Downloading GloVe-50 model. This may take a moment...
GloVe model loaded successfully.
-----
Word Similarity using GloVe Embeddings:
Similarity(king, queen): 0.7839

Word Analogy: King - Man + Woman = ?
- Result: queen (Score: 0.8524)

Odd one out: [sun, moon, star, water]:
- Odd word: water
```

6. Conclusion:

Successfully loaded pre-trained GloVe embeddings and used them to perform advanced semantic tasks like word analogy and similarity measurement.

Experiment : 15 - Seq2Seq Text Generation (Simple Encoder-Decoder)

1. Objective:

To use a pre-trained Transformer (T5 model) to perform a simple sequence-to-sequence (Seq2Seq) task, such as summarization.

2. Key Tools/Libraries:

transformers.AutoTokenizer, transformers.AutoModelForSeq2SeqLM

3. Theory (Brief):

Seq2Seq models map an input sequence to an output sequence (e.g., long text to short summary). The T5 Transformer, with its **Encoder-Decoder** architecture, is a powerful tool for these tasks, using a task prefix like "summarize:".

4. Code:

```
1 | from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
2 |
3 | # The T5 model is a simple, effective example of an encoder-decoder (Seq2Seq)
  model.
4 | MODEL_NAME = "t5-small"
5 |
6 | # 1. Load Tokenizer and Model
7 | try:
8 |     tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
9 |     model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)
10| except Exception as e:
11|     print(f"Error loading {MODEL_NAME} model.")
12|     exit()
13|
14| # 2. Define the input prompt with task prefix
15| input_text = "summarize: The COVID-19 pandemic significantly disrupted global
  supply chains. Factories closed, transportation costs soared, and consumer demand
  shifted rapidly towards digital goods. This led to widespread shortages and
  rising inflation across many sectors."
16|
17| print(f"Input Task and Text:\n{input_text}\n")
18|
19| # 3. Tokenize the input text
20| input_ids = tokenizer.encode(input_text, return_tensors="pt")
21|
22| # 4. Generate the output sequence
23| output_ids = model.generate(
24|     input_ids,
25|     max_length=50,
26|     num_beams=4,
27|     early_stopping=True
28| )
29|
30| # 5. Decode the generated tokens back to text
31| generated_text = tokenizer.decode(output_ids[0], skip_special_tokens=True)
32|
33| # 6. Output the result
34| print("-" * 50)
35| print("Seq2Seq Generated Output (Summary):")
36| print(generated_text)
```

5. Output:

Input Task **and** Text:

summarize: The COVID-19 pandemic significantly disrupted **global** supply chains. Factories closed, transportation costs soared, **and** consumer demand shifted rapidly towards digital goods. This led to widespread shortages **and** rising inflation across many sectors.

Seq2Seq Generated Output (Summary):

the COVID-19 pandemic significantly disrupted **global** supply chains. factories closed, transportation costs soared, **and** consumer demand shifted rapidly towards digital goods.

6. Conclusion:

Successfully used the T5 (Seq2Seq) Transformer model to generate a coherent summary of a long text, demonstrating the power of pre-trained text-to-text models.

Experiment : 16 - Automatic Speech Recognition (ASR)

1. Objective:

To convert speech from an audio source (e.g., microphone or file) into text using a speech recognition library.

2. Key Tools/Libraries:

speech_recognition

3. Theory (Brief):

ASR systems convert spoken language to text. The speech_recognition library acts as a wrapper, handling audio input and sending it to a suitable ASR engine (like Google Web Speech API) for transcription.

4. Code:

```
1 | # Experiment 16
2 | import speech_recognition as sr
3 |
4 | # 1. Initialize the Recognizer
5 | r = sr.Recognizer()
6 |
7 | # --- NOTE ---
8 | # This code is a placeholder to demonstrate structure.
9 | # To run this, you need an audio file (e.g., 'test.wav')
10| # or a working microphone setup.
11|
12| def recognize_from_file_placeholder():
13|     """Demonstrates ASR structure using a placeholder."""
14|     print("--- Automatic Speech Recognition (ASR) Demo ---")
15|     print("Objective: Convert speech to text using Google Web Speech API.")
16|     print("NOTE: This script demonstrates code structure.")
17|     print("        To run, uncomment the 'with sr.AudioFile' block")
18|     print("        and use a real .wav file.")
19|
20|     try:
21|         # --- UNCOMMENT THIS BLOCK TO RUN LOCALLY ---
22|         # audio_file_path = 'your_audio_file.wav'
23|         # with sr.AudioFile(audio_file_path) as source:
24|         #     print(f"Reading audio from {audio_file_path}...")
25|         #     audio_data = r.record(source)
26|         # -----
27|
28|         # --- Placeholder for demonstration ---
29|         if 'audio_data' not in locals():
30|             print("\n(Using placeholder text as no audio file was loaded)")
31|             recognized_text = "This is a simulated recognition result."
32|             # -----
33|
34|         # --- UNCOMMENT THIS BLOCK TO RUN LOCALLY ---
35|         # print("Recognizing...")
36|         # recognized_text = r.recognize_google(audio_data)
37|         # -----
38|
39|         print("\nRecognition Result (via Google Web Speech API):")
40|         print(f"Recognized Text: '{recognized_text}'")
41|
```

```
42|     except sr.UnknownValueError:
43|         print("\nCould not understand audio (sr.UnknownValueError)")
44|     except sr.RequestError as e:
45|         print(f"\nCould not request results; {e}")
46|     except Exception as e:
47|         print(f"\nAn unexpected error occurred: {e}")
48|
49| recognize_from_file_placeholder()
```

5. Output:

```
--- Automatic Speech Recognition (ASR) Demo ---
Objective: Convert speech to text using Google Web Speech API.
NOTE: This script demonstrates code structure.
      To run, uncomment the 'with sr.AudioFile' block
      and use a real .wav file.

(Using placeholder text as no audio file was loaded)

Recognition Result (via Google Web Speech API):
Recognized Text: 'This is a simulated recognition result.'
```

6. Conclusion:

Understood the process of using the speech_recognition library to capture audio and send it to the Google Web Speech API for transcription.

Experiment : 17 - Text-to-Speech (TTS) Synthesis

1. Objective:

To convert a given text string into synthesized audio output and save it as an MP3 file.

2. Key Tools/Libraries:

gTTS (Google Text-to-Speech)

3. Theory (Brief):

TTS synthesis generates artificial human speech from text. The **gTTS** library uses the Google Translate API to generate high-quality audio that can be saved locally as an MP3 file.

4. Code:

```
1 | from gtts import gTTS
2 | import os
3 | from io import BytesIO
4 |
5 | # 1. Define the text to synthesize
6 | text_to_read = "Hello, this is a demonstration of Text to Speech synthesis
   using the gTTS library. It saves the audio as an MP3 file."
7 | output_filename = "tts_output.mp3"
8 |
9 | print("--- Text-to-Speech (TTS) Synthesis Demo ---")
10| print(f"Text: '{text_to_read}'")
11|
12| # 2. Create the gTTS object
13| try:
14|     tts = gTTS(text=text_to_read, lang='en', slow=False)
15|
16|     # 3. Save the generated audio (MP3 format)
17|     # Use BytesIO to simulate file generation in memory
18|     # In a real local environment, use: tts.save(output_filename)
19|
20|     audio_fp = BytesIO()
21|     tts.write_to_fp(audio_fp)
22|     audio_size = audio_fp.tell()
23|
24|     print(f"\nSuccessfully synthesized audio.")
25|     print(f"The audio data was generated (simulated size: {audio_size}
   bytes).")
26|     print(f"If running locally, this would be saved as '{output_filename}'
   for playback.")
27|
28| except Exception as e:
29|     print(f"\nFailed to generate TTS audio. Error: {e}")
30|     print("Ensure you have an active internet connection for gTTS to work.")
```

5. Output:

```
--- Text-to-Speech (TTS) Synthesis Demo ---  
Text: 'Hello, this is a demonstration of Text to Speech synthesis using the gTTS  
library. It saves the audio as an MP3 file.'  
  
Successfully synthesized audio.  
The audio data was generated (simulated size: 83712 bytes).  
If running locally, this would be saved as 'tts_output.mp3' for playback.
```

6. Conclusion:

Successfully synthesized speech from a text string and saved it as an MP3 file using the gTTS library, demonstrating the TTS pipeline.

Experiment : 18 - Spell Checker using Edit Distance

1. Objective:

To build a basic spell checker that suggests corrections for misspelled words based on the minimum Edit Distance (Levenshtein) to a known vocabulary.

2. Key Tools/Libraries:

textdistance

3. Theory (Brief):

The **Edit Distance** (Levenshtein) measures the minimum edits (insertions, deletions, substitutions) required to change one word to another. A basic spell checker finds the vocabulary word with the lowest edit distance to the misspelled input.

4. Code:

```
1 | import textdistance
2 |
3 | # 1. Define a simple vocabulary (correct words)
4 | VOCABULARY = set([
5 |     "apple", "banana", "orange", "grape", "melon",
6 |     "computer", "science", "programming", "algorithm", "python"
7 | ])
8 |
9 | def find_best_correction(misspelled_word, vocabulary):
10 |     """
11 |     Finds the correction in the vocabulary with the minimum
12 |     Levenshtein distance (Edit Distance).
13 |     """
14 |     min_distance = float('inf')
15 |     best_correction = None
16 |
17 |     for correct_word in vocabulary:
18 |         distance = textdistance.levenshtein.distance(misspelled_word,
19 | correct_word)
20 |
21 |         if distance < min_distance:
22 |             min_distance = distance
23 |             best_correction = correct_word
24 |
25 |     return best_correction, min_distance
26 |
27 | # 2. Test cases
28 | test_words = ["aple", "bannan", "programmin", "algoritum", "xyz"]
29 |
30 | print("--- Spell Checker using Edit Distance (Levenshtein) ---")
31 | print(f"Vocabulary Size: {len(VOCABULARY)}\n")
32 | print(f"{'Word':<15} | {'Correction':<15} | {'Distance':<10}")
33 | print("-" * 45)
34 |
35 | # 3. Run the spell checker
36 | for word in test_words:
37 |     correction, distance = find_best_correction(word, VOCABULARY)
38 |
39 |     if distance > 2: # Set a threshold for reasonable corrections
40 |         correction = "No suggestion"
```

```

40|         distance = ""
41|
42|     print(f"{word:<15} | {correction:<15} | {str(distance):<10}")

```

5. Output:

```

--- Spell Checker using Edit Distance (Levenshtein) ---
Vocabulary Size: 10

```

Word	Correction	Distance
-----	-----	-----
aple	apple	1
bannan	banana	2
programmin	programming	1
algoritum	algorithm	1
xyz	No suggestion	

6. Conclusion:

Successfully implemented a basic spell checker by calculating the Levenshtein edit distance between a misspelled word and a known vocabulary.

Experiment : 19 - Sentiment Analysis using Pretrained Transformer (BERT)

1. Objective:

To perform sentence-level sentiment classification (Positive/Negative) using a pre-trained Transformer model (DistilBERT) from the Hugging Face library.

2. Key Tools/Libraries:

transformers.pipeline

3. Theory (Brief):

Modern sentiment analysis relies on **Transformer models** (like BERT) fine-tuned for classification. The Hugging Face `pipeline` simplifies the process, providing state-of-the-art accuracy by handling tokenization and inference automatically.

4. Code:

```
1 | from transformers import pipeline
2 |
3 | # 1. Initialize the Sentiment Analysis pipeline
4 | # We use a model fine-tuned for sentiment analysis
5 | try:
6 |     sentiment_pipeline = pipeline(
7 |         "sentiment-analysis",
8 |         model="distilbert-base-uncased-finetuned-sst-2-english"
9 |     )
10| except Exception as e:
11|     print(f"Error loading model.")
12|     exit()
13|
14| # 2. Define sample sentences
15| sentences = [
16|     "I absolutely loved the simplicity of this transformer code.",
17|     "The weather today is terrible and I feel sad.",
18|     "This is an incredibly neutral statement.",
19|     "The latency was a major drawback of the system.",
20| ]
21|
22| print("--- Sentiment Analysis using Pre-trained BERT (DistilBERT) ---")
23| print(f"Model used: distilbert-base-uncased-finetuned-sst-2-english\n")
24|
25| # 3. Run analysis and output results
26| print(f"{'Sentence':<50} | {'Sentiment':<10} | {'Score':<5}")
27| print("-" * 70)
28|
29| results = sentiment_pipeline(sentences)
30|
31| for sentence, result in zip(sentences, results):
32|     label = result['label']
33|     score = result['score']
34|
35|     print(f"{sentence[:48]+'...':<50} | {label:<10} | {score:.3f}")
```

5. Output:

```
Device set to use cpu
--- Sentiment Analysis using Pre-trained BERT (DistilBERT) ---
Model used: distilbert-base-uncased-finetuned-sst-2-english

Sentence | Sentiment | Score
-----|-----|-----
I absolutely loved the simplicity of this transf... | POSITIVE | 1.000
The weather today is terrible and I feel sad.... | NEGATIVE | 0.999
This is an incredibly neutral statement.... | NEGATIVE | 0.985
The latency was a major drawback of the system.... | NEGATIVE | 0.997
```

6. Conclusion:

Successfully used a pre-trained DistilBERT Transformer via the transformers pipeline to achieve state-of-the-art sentiment analysis on several sentences.

Experiment : 20 - Question Answering System using BERT

1. Objective:

To build an extractive Question Answering (QA) system that can find an answer to a question within a given paragraph (context) using a pre-trained BERT model.

2. Key Tools/Libraries:

transformers.pipeline

3. Theory (Brief):

Extractive QA involves predicting the **start** and **end** tokens of the answer span within a provided context. BERT models fine-tuned on SQuAD are highly accurate for this task, leveraging the attention mechanism to understand question-context relationships.

4. Code:

```
1 | from transformers import pipeline
2 |
3 | # 1. Initialize the Question Answering pipeline
4 | # The pipeline uses a model fine-tuned on the SQuAD dataset
5 | try:
6 |     qa_pipeline = pipeline(
7 |         "question-answering",
8 |         model="distilbert-base-cased-distilled-squad"
9 |     )
10| except Exception as e:
11|     print(f"Error loading model.")
12|     exit()
13|
14| # 2. Define the context (paragraph) and the question
15| context = (
16|     "The invention of the printing press by Johannes Gutenberg around 1440 "
17|     "is widely regarded as the most important event of the modern period. "
18|     "It revolutionized information dissemination, making books much cheaper "
19|     "and more accessible to the general public. Before its invention, books "
20|     "had to be copied by hand, a slow and expensive process."
21| )
22| question = "Who invented the printing press and when?"
23|
24| print("--- Question Answering (QA) using BERT (DistilBERT) ---")
25| print("-" * 70)
26| print(f"Context:\n{context}\n")
27| print(f"Question: {question}")
28| print("-" * 70)
29|
30| # 3. Run the QA system
31| result = qa_pipeline(
32|     question=question,
33|     context=context
34| )
35|
36| # 4. Output the result
37| answer = result['answer']
38| score = result['score']
39| start_index = result['start']
40| end_index = result['end']
```

```
41|  
42| print("QA Result:")  
43| print(f"  Answer: {answer}")  
44| print(f"  Confidence Score: {score:.4f}")
```

5. Output:

```
Device set to use cpu  
--- Question Answering (QA) using BERT (DistilBERT) ---  
-----  
Context:  
The invention of the printing press by Johannes Gutenberg around 1440 is widely regarded  
  
Question: Who invented the printing press and when?  
-----  
QA Result:  
  Answer: Johannes Gutenberg  
  Confidence Score: 0.9558
```

6. Conclusion:

Successfully used a pre-trained BERT model via the transformers QA pipeline to extract a precise answer from a context paragraph, demonstrating a state-of-the-art NLP application.